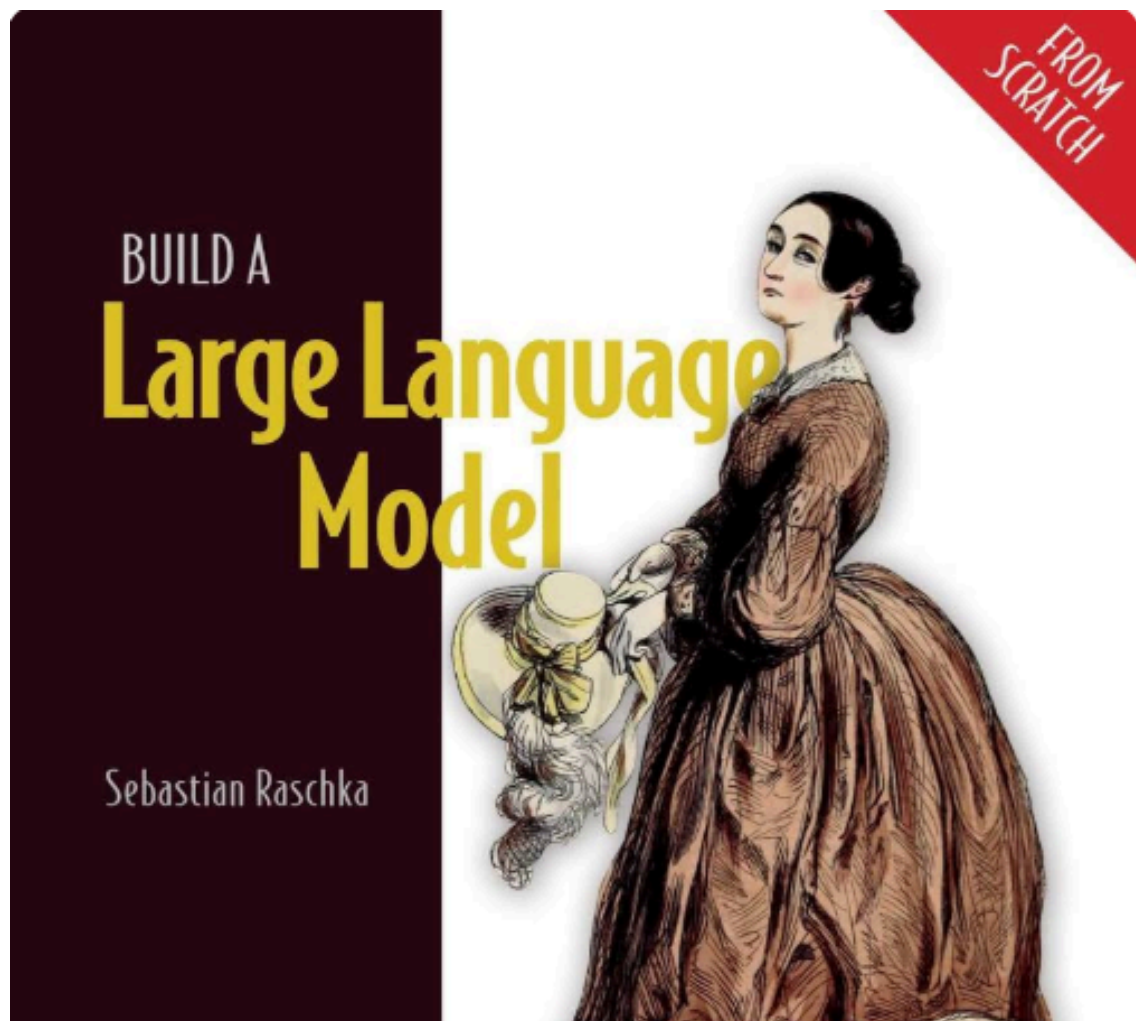


GLOSSÁRIO

Agosto 2026

Vitória Aparecida Vendausen

Matheus Dapper



Glossário

GLOSSÁRIO	0
1.Capitulo 1	3
2.Capitulo 2	5
	8
	10
	11
	11
	11
	11
	12
	12
	12
	14
	15

1. Capítulo 1

Reinforcement Learning from Human Feedback (RLHF)

Português: Aprendizado por reforço com feedback humano (*substantivo masculino*)

- **Definição:** Esse tipo de otimização funciona com especialistas humanos avaliando as respostas do modelo para demonstrar o que é um conteúdo seguro, verdadeiro e útil.
- **Função no Modelo:** O sistema propõe um texto para que um treinador humano julgue ou escolha a melhor resposta para recompensar o algoritmo, guiando o modelo para longe de boatos, preconceitos ou piadas indesejadas.
- **Relação com Outros Conceitos:** Geralmente é o método utilizado para realizar o alinhamento do modelo após a conclusão da etapa de ajuste fino.
- **Exemplo:** O ChatGPT propõe três textos possíveis para o usuário e um revisor humano pontua a melhor opção, fazendo com que o sistema aprenda a priorizá-la no futuro.

Supervised Learning

Português: Aprendizado supervisionado (*substantivo masculino*)

- **Definição:** O paradigma tradicional de treinamento no qual um algoritmo aprende comparando suas palpites a um conjunto de respostas fornecido junto com os dados de treino.
- **Função no Modelo:** Ao analisar milhares de exemplos pareados, o sistema aprende a executar uma ação sobre um texto para obter outro texto desejado.
- **Relação com Outros Conceitos:** Serve como base para modelos de instrução direta ou etapas de ajuste fino.
- **Exemplo:** Mostrar ao algoritmo quais e-mails foram classificados por pessoas como "Spam" ou "Caixa de Entrada", permitindo que ele aprenda a executar essa mesma tarefa sozinho.

Self-Supervised Learning

Português: Aprendizado auto-supervisionado (*substantivo masculino*)

- **Definição:** Um conjunto de métodos em que o algoritmo se treina em dados não rotulados, dispensando a necessidade de anotações humanas.
- **Função no Modelo:** Permite que o sistema analise milhares de páginas da web para aprender conceitos gerais de linguagem por conta própria.

- **Relação com Outros Conceitos:** A técnica fundamental que constitui toda a etapa de pré-treinamento.
- **Exemplo:** Pedir para o modelo adivinhar a palavra "azul" na frase "O céu está muito ____ hoje", utilizando o restante da internet como dado de treinamento.

Fine-Tuning

Português: Ajuste fino (*substantivo masculino*)

- **Definição:** O ajuste dos pesos de um modelo para especializá-lo em uma tarefa específica, expondo-o a um conjunto mais restrito de informações.
- **Função no Modelo:** Permite atualizar um modelo de linguagem genérico e pré-treinado para torná-lo melhor em uma tarefa específica sem desperdiçar recursos.
- **Relação com Outros Conceitos:** Geralmente posicionado entre o pré-treinamento e a implantação final, funcionando como um caminho para criar ferramentas úteis a partir de modelos grandes e genéricos.
- **Exemplo:** Expor uma IA de uso geral a milhares de documentos jurídicos para capacitá-la a redigir peças do Direito.

Alignment

Português: Alinhamento (*substantivo masculino*)

- **Definição:** O ajuste do comportamento do modelo para atender às expectativas humanas, evitando a geração de conteúdos prejudiciais.
- **Função no Modelo:** Filtra as respostas do sistema para impedir que informações tóxicas, perigosas ou tabus sejam geradas.
- **Relação com Outros Conceitos:** Geralmente realizado por meio do aprendizado por reforço com feedback humano (RLHF) para moldar a personalidade de um assistente virtual.
- **Exemplo:** Recompensar o modelo por se recusar a responder "Como invadir um sistema" e penalizá-lo caso ele forneça dicas úteis de invasão.

Hallucination

Português: Alucinação (*substantivo masculino*)

- **Definição:** A produção de informações não encontradas nos dados de treinamento, frequentemente apresentadas como fatos ou citações reais.
- **Função no Modelo:** Um risco inerente ao sistema de previsão da próxima palavra, que faz a IA dar palpites além das informações concretas que possui.
- **Relação com Outros Conceitos:** Frequentemente ligada a configurações de alta temperatura e à tarefa de previsão da próxima palavra.

- **Exemplo:** Pedir o resumo de um livro que nunca foi publicado e receber uma resposta longa, com direito a citações fabricadas e uma data de lançamento falsa.

Transformer Architecture

Português: Arquitetura Transformer (*substantivo feminino*)

- **Definição:** Um tipo de rede neural que processa textos analisando todas as palavras simultaneamente, ao contrário do processamento sequencial antigo.
- **Função no Modelo:** Permite realizar cálculos em grandes volumes de texto de uma só vez e conectar ideias separadas por vários parágrafos.
- **Relação com Outros Conceitos:** Substituiu as arquiteturas recorrentes antigas e viabilizou os modelos de base modernos.
- **Exemplo:** A tecnologia fundamental por trás da maioria dos assistentes modernos, como ChatGPT, Gemini ou Claude.

Selective Attention

Português: Atenção seletiva (*substantivo feminino*)

- **Definição:** O mecanismo pelo qual a rede neural foca mais em palavras específicas para compreender o seu contexto real.
- **Função no Modelo:** Permite ao modelo associar determinadas expressões para descartar significados irrelevantes em um dado contexto.
- **Relação com Outros Conceitos:** Geralmente implementada através do mecanismo de autoatenção.
- **Exemplo:** Diante da frase "O banco da praça estava molhado", o sistema entende que a palavra "banco" se refere ao assento, descartando o sentido de instituição financeira.

Autoregression

Português: Autoregressão (*substantivo feminino*)

- **Definição:** Uma técnica que permite ao modelo escrever textos prevendo uma palavra de cada vez.
- **Função no Modelo:** Auxilia o modelo a gerar textos longos e coerentes, utilizando a última palavra digitada como ponto de partida para a próxima.
- **Relação com Outros Conceitos:** Habilita o comportamento de predição da próxima palavra na parte decodificadora do modelo.
- **Exemplo:** No momento em que o sistema digita "O", ele calcula que "sol" é mais provável que "céu" e, após "O sol", projeta o verbo "brilha".

Emergent Behavior

Português: Capacidade emergente (*substantivo feminino*)

- **Definição:** O surgimento de novas habilidades e competências que não foram treinadas de forma explícita.
- **Função no Modelo:** Torna o sistema capaz de realizar tarefas surpreendentes que não lhe foram ensinadas diretamente.
- **Relação com Outros Conceitos:** Geralmente surge em modelos de linguagem de grande escala devido ao aumento no volume de dados e na quantidade de parâmetros.
- **Exemplo:** Um modelo de linguagem projetado para prever a próxima palavra conseguir resolver problemas matemáticos complexos e traduzir poesias antigas.

Encoder

Português: Codificador (*substantivo masculino*)

- **Definição:** A parte da rede neural cujo propósito é compreender o contexto e o significado de um determinado texto.
- **Função no Modelo:** Ideal para tarefas que exigem análise ou compreensão profunda, em vez da escrita criativa.
- **Relação com Outros Conceitos:** Geralmente combinado com a parte decodificadora em modelos do tipo Codificador-Decodificador (*Encoder-Decoder*).
- **Exemplo:** Um sistema automatizado que lê avaliações de clientes e as separa em grupos de comentários positivos ou negativos.

Zero-Shot Learning

Português: Comportamento de nenhuma amostra (*substantivo masculino*)

- **Definição:** A capacidade de realizar uma tarefa sem ter sido explicitamente treinado para ela.
- **Função no Modelo:** Permite que o sistema siga instruções que não faziam parte dos seus dados originais de treinamento.
- **Relação com Outros Conceitos:** Pode ser visto como uma forma de capacidade emergente.
- **Exemplo:** Pedir para escrever um poema sobre o mar no estilo de Fernando Pessoa e obter uma resposta válida sem fornecer nenhum contexto adicional.

Few-Shot Learning

Português: Comportamento de poucas amostras (*substantivo masculino*)

- **Definição:** A capacidade de deduzir regras gerais para uma tarefa a partir de poucos exemplos incluídos junto com a própria requisição.
- **Função no Modelo:** Permite ao sistema adaptar seu comportamento a diferentes tarefas sem a necessidade de treinamentos adicionais.
- **Relação com Outros Conceitos:** Funciona como um meio-termo entre o aprendizado sem amostras (*zero-shot*) e o ajuste fino (*fine-tuning*).
- **Exemplo:** Pedir para converter frases em gírias fornecendo alguns exemplos no comando e obter resultados corretos para os pedidos seguintes.

Decoder

Português: Decodificador (*substantivo masculino*)

- **Definição:** A parte da rede neural focada na geração de textos fluídos e coerentes.
- **Função no Modelo:** Permite ao sistema continuar escrevendo ao analisar o contexto e prever as próximas palavras mais prováveis.
- **Relação com Outros Conceitos:** Geralmente aparece ao lado do codificador ou é utilizado de forma autônoma em modelos autorregressivos.
- **Exemplo:** O componente responsável por fazer o ChatGPT continuar gerando novo texto de forma sequencial na sua tela.

Embeddings

Português: Vetores de incorporação / Embeddings (*substantivo masculino plural*)

- **Definição:** A associação de palavras ou conceitos a coordenadas numéricas dentro de um espaço multidimensional.
- **Função no Modelo:** Permite comparar o grau de parentesco e o significado entre diferentes palavras ou frases.
- **Relação com Outros Conceitos:** Geralmente calculado pela etapa codificadora do modelo.
- **Exemplo:** O sistema entender que a palavra "rainha" está mais próxima de "rei" do que de "moeda" ao analisar suas coordenadas numéricas.

Manual Feature Extraction

Português: Extração manual de características (*substantivo feminino*)

- **Definição:** Formato antigo de programação onde especialistas humanos precisavam definir manualmente o que o computador deveria procurar.
- **Função no Modelo:** Útil para tarefas muito específicas, porém limitado pelo conhecimento do seu criador.

- **Relação com Outros Conceitos:** O método fundamental que foi gradualmente substituído pelo aprendizado auto-supervisionado.
- **Exemplo:** Escrever regras de código estritas determinando como classificar e-mails recebidos como spam ou não.

Prompt

Português: Instrução de entrada / Prompt (*substantivo feminino*)

- **Definição:** Um comando ou pergunta feita à IA para que ela execute uma tarefa específica.
- **Função no Modelo:** A instrução direta que o usuário digita para descrever o que deseja que a IA faça.
- **Relação com Outros Conceitos:** Pode ser utilizado para definir exemplos de poucas amostras (*few-shot*) ou comandos diretos sem amostras (*zero-shot*).
- **Exemplo:** Pedir para resumir a Revolução Francesa em três tópicos simples para uma criança.

Generative Artificial Intelligence (GenAI)

Português: Inteligência artificial generativa (*substantivo feminino*)

- **Definição:** Sistemas modernos de IA capazes de gerar novos textos, áudios, imagens ou códigos de computador através da análise de padrões existentes.
- **Função no Modelo:** Permite ao sistema ir além das tarefas puramente linguísticas e entrar na criação ativa de mídias.
- **Relação com Outros Conceitos:** Os modelos de base são geralmente fundamentados em tecnologias de IA generativa.
- **Exemplo:** Pedir para criar a imagem de um astronauta andando a cavalo em Marte e receber uma pintura detalhada em resposta.

Context Window

Português: Janela de contexto (*substantivo feminino*)

- **Definição:** A quantidade de texto que o modelo consegue analisar de uma só vez para entender o que o usuário está solicitando.
- **Função no Modelo:** Determina o volume máximo de texto que pode ser processado ou lembrado em um dado momento.
- **Relação com Outros Conceitos:** Impõe o limite operacional para que o mecanismo de autoatenção consiga acompanhar uma conversa longa.
- **Exemplo:** A capacidade de colar um livro inteiro no chat para que o modelo analise e responda a perguntas sobre ele.

Self-Attention Mechanism

Português: Mecanismo de autoatenção (*substantivo masculino*)

- **Definição:** O processo pelo qual o modelo compara cada palavra de uma frase com todas as outras palavras da mesma sentença.
- **Função no Modelo:** Mantém o rastreio de todos os termos utilizados para que a máquina não se esqueça das partes iniciais de uma frase longa.
- **Relação com Outros Conceitos:** O mecanismo fundamental que permite à arquitetura Transformer processar frases inteiras de uma só vez.
- **Exemplo:** Na frase "A menina que morava na casa amarela no fim da rua comprou um livro", o sistema associa a primeira palavra "A" (referente à menina) diretamente ao verbo "comprou".

Foundation Model

Português: Modelo de base (*substantivo masculino*)

- **Definição:** Sistemas de IA de altíssimo porte que fornecem uma compreensão ampla e de uso geral sobre o mundo.
- **Função no Modelo:** Atua como a fundação primária para a maioria das aplicações práticas de IA generativa.
- **Relação com Outros Conceitos:** Geralmente passa por ajustes finos (*fine-tuning*) para originar ferramentas e funções específicas.
- **Exemplo:** Grandes modelos de linguagem, como os desenvolvidos pela OpenAI ou Meta, que sustentam diversas aplicações de mercado.

Large Language Model (LLM)

Português: Modelo de linguagem de grande porte (*substantivo masculino*)

- **Definição:** Um tipo de rede neural que utiliza bilhões de conexões internas para compreender as linguagens humanas.
- **Função no Modelo:** Atua como um "motor de raciocínio" de uso geral para tarefas de linguagem natural.
- **Relação com Outros Conceitos:** A aplicação prática mais conhecida da arquitetura Transformer na área da IA generativa.
- **Exemplo:** A tecnologia fundamental que permite o funcionamento de assistentes como Claude, Gemini ou ChatGPT.

Parameter

Português: Parâmetro (*substantivo masculino*)

- **Definição:** Uma engrenagem interna no algoritmo que é calibrada para reduzir os erros do modelo durante o aprendizado.
- **Função no Modelo:** Utilizado para armazenar as informações e o conhecimento consolidados pela máquina.
- **Relação com Outros Conceitos:** Maior quantidade de parâmetros geralmente resulta em maior precisão e capacidade emergente.
- **Exemplo:** Um modelo com 70 bilhões de parâmetros utiliza 70 bilhões dessas "engrenagens" numéricas para calcular suas respostas.

Next-Word Prediction

Português: Predição da próxima palavra (*substantivo feminino*)

- **Definição:** A tarefa fundamental utilizada no treinamento de modelos de linguagem, calculando a probabilidade de cada palavra sequencial.
- **Função no Modelo:** Permite ao sistema aprender as regras básicas de gramática e a estrutura da linguagem.
- **Relação com Outros Conceitos:** A base para a maioria dos modelos autorregressivos.
- **Exemplo:** Dado o início de uma frase, o sistema calcula que a próxima palavra é muito mais provável de ser "muro" do que "oceano".

Masked Word Prediction

Português: Predição de palavras mascaradas (*substantivo feminino*)

- **Definição:** Uma metodologia de treinamento onde palavras são propositalmente ocultadas dentro de uma frase, estimulando a IA a adivinhar o conteúdo ausente analisando o contexto à esquerda e à direita.
- **Função no Modelo:** Força a IA a desenvolver uma visão completa e bidirecional da estrutura das frases.
- **Relação com Outros Conceitos:** A principal abordagem de treinamento utilizada em modelos codificadores.
- **Exemplo:** Fornecer a sequência "O [MASCARADO] latiu para o carteiro" para que o algoritmo observe as palavras vizinhas e deduza que o termo oculto é "cachorro".

Pretraining

Português: Pré-treinamento (*substantivo masculino*)

- **Definição:** Marca a fase inicial e mais onerosa da construção de um modelo, onde o algoritmo consome repositórios massivos da web para aprender como os humanos se comunicam.

- **Função no Modelo:** Constrói a base de conhecimento geral do mundo e a infraestrutura linguística antes de qualquer especialização.
- **Relação com Outros Conceitos:** Antecede o ajuste fino (*fine-tuning*) e utiliza o aprendizado auto-supervisionado como seu método principal.
- **Exemplo:** Executar clusters de supercomputadores por meses para processar portais de notícias, livros digitais e artigos da Wikipédia para que a rede aprenda a redigir.

Natural Language Processing (NLP)

Português: Processamento de linguagem natural (*substantivo masculino*)

- **Definição:** O subcampo da ciência da computação e da IA focado em aproximar a comunicação entre humanos e máquinas, unindo computação avançada e linguística.
- **Função no Modelo:** Capacita os computadores a ler, interpretar, analisar e gerar textos humanos da forma mais natural possível.
- **Relação com Outros Conceitos:** O campo acadêmico e tecnológico amplo que deu origem às redes neurais e aos LLMs.
- **Exemplo:** Ferramentas de acessibilidade que convertem texto em fala, plataformas de tradução instantânea ou corretores gramaticais inteligentes.

Deep Neural Network

Português: Rede neural profunda (*substantivo feminino*)

- **Definição:** Um modelo de software livremente inspirado no cérebro biológico, composto por múltiplas camadas digitais de processamento que trabalham juntas para resolver problemas complexos.
- **Função no Modelo:** Serve para fatiar dados complexos, com cada camada limpando, filtrando e interpretando uma parte da informação de entrada.
- **Relação com Outros Conceitos:** A infraestrutura algorítmica que viabiliza o aprendizado profundo (*deep learning*).
- **Exemplo:** Uma camada reconhece letras individuais, a seguinte forma palavras, a terceira capta o sentido da frase e a camada mais profunda identifica a intenção do usuário.

Temperature

Português: Temperatura (*substantivo feminino*)

- **Definição:** Uma configuração de controle matemático que ajusta o quão ousado, criativo ou imprevisível o modelo pode ser ao selecionar a próxima palavra.
- **Função no Modelo:** Permite alternar o comportamento da máquina entre um modo preciso e determinístico ou um modo mais solto e criativo.

- **Relação com Outros Conceitos:** Altera diretamente a distribuição de probabilidades durante a predição da próxima palavra.
- **Exemplo:** Configurar a temperatura em 0.1 para obter um relatório financeiro exato, ou subir para 0.9 para gerar ideias criativas de nomes de marcas.

Tokenization

Português: Tokenização (*substantivo feminino*)

- **Definição:** O processo técnico de quebrar um texto bruto em pedaços menores (chamados *tokens*, como palavras ou fragmentos de palavras) e convertê-los em códigos numéricos.
- **Função no Modelo:** Traduz a linguagem humana em um formato numérico que os processadores conseguem calcular.
- **Relação com Outros Conceitos:** O primeiro passo no processamento de texto, ocorrendo antes dos vetores de incorporação (*embeddings*) e dos mecanismos de autoatenção.
- **Exemplo:** Divisão da palavra "Inacreditável" em fragmentos menores ["In", "acreditá", "vel"] e repasse da sequência numérica correspondente para a máquina.

Capítulo 2:

Glossário de Termos — LLMs

Tokenização e Embeddings (Capítulo 2)

Token

Tradução: Token.

O que é: Cada um dos pedaços de texto gerados pelo tokenizador (uma palavra, parte dela ou um caractere especial).

Função no LLM: É a unidade discreta básica que carrega a informação linguística para dentro do modelo.

Relação com outros conceitos: É a chave para buscar os Token IDs correspondentes na tabela do Vocabulário.

Exemplo do Livro

```
# Uma lista simples de strings contendo tokens extraídos do conto "The Verdict"
tokens_livro = [
    'I', 'HAD', 'always', 'thought', 'Jack', 'Gisburn', 'rather',
    'a', 'cheap', 'genius', '--', 'though', 'a', 'good', 'fellow'
]
```

O que o código do livro está fazendo: Aqui temos uma lista puramente textual em Python. Cada elemento (como 'I', 'always', e o travessão '--') é um token independente obtido a partir da quebra do texto original. O modelo agora enxerga a frase como uma sequência de elementos isolados.

Exemplo Criado

```
# A palavra "correndo" dividida em dois tokens (radical + sufixo):  
tokens_subpalavra = ['corr', 'endo']
```

Identificador de Token (Token ID)

Tradução: ID do Token.

O que é: Um número inteiro exclusivo associado a cada palavra/símbolo do vocabulário.

Função no LLM: Substitui o caractere textual por um número inteiro simples para que a rede neural possa usar esse número como índice de busca na memória.

Relação com outros conceitos: Representa a tradução do Token em número através do Vocabulário.

Exemplo do Livro

```
# Lista de números inteiros que representam a frase indexada  
token_ids_livro = [1, 56, 2, 850, 988, 602, 533, 746, 5, 1126, 596, 5]
```

O que o código do livro está fazendo: Esta lista de inteiros substitui diretamente o texto original. Cada número aponta para o índice da palavra correspondente no dicionário do vocabulário do livro. Por exemplo, o ID 1 é o caractere " ", 56 é 'It', e 2 é 's'.

Exemplo Criado

```
# Em um sistema simplificado de controle escolar:  
# A palavra 'presença' está no índice 5, logo: Token ID = 5
```

Vocabulário (Vocabulary)

Tradução: Vocabulário.

O que é: Um dicionário (tabela de mapeamento) que contém todas as palavras que o modelo conhece e seus respectivos IDs.

Função no LLM: Funciona como o dicionário de idiomas do modelo. Define quais palavras a rede neural consegue compreender.

Relação com outros conceitos: Faz a ponte de tradução bidirecional entre Tokens e Token IDs.

Exemplo do Livro

```
# Dicionário de mapeamento conceitual construído a partir do texto  
vocab_livro = {  
    '!': 0,  
    '"': 1,  
    "'": 2,  
    # ... (índices intermediários) ...
```

```
'Her': 49,  
'Hermia': 50,  
# ...  
'yourself': 1129  
}
```

O que o código do livro está fazendo: Este dicionário em Python mapeia cada palavra exclusiva (chave) para um número inteiro único (valor). O livro ordena as palavras em ordem alfabética para definir a numeração: pontuações como ! e " ganham os IDs iniciais (0 e 1), palavras com letras maiúsculas como 'Her' e 'Hermia' vêm no meio, e letras minúsculas vêm no final.

Exemplo Criado

```
# Um vocabulário ultra simplificado de três itens:  
meu_vocab = {"o": 0, "gato": 1, "miou": 2}
```

Palavra Desconhecida (Out-of-Vocabulary - OOV)

Tradução: Fora do Vocabulário.

O que é: Um termo que não apareceu nos dados de treino e, portanto, não existe no vocabulário do modelo.

Função no LLM: Evita que o programa quebre ao ver uma palavra nova, forçando o uso de um token substituto padrão.

Relação com outros conceitos: É a falha de busca no Vocabulário, resolvida pelo BPE ou pelo uso de <|unk|>.

Exemplo do Livro

```
# Tentativa de buscar um termo inexistente na tabela  
vocab_livro = {"I": 0, "thought": 1, "Jack": 2} # Exemplo de dicionário curto  
try:  
    vocab_livro["Hello"]  
except KeyError as e:  
    print(f"Erro gerado: KeyError: {e}")  
# Saída no terminal: Erro gerado: KeyError: 'Hello'
```

O que o código do livro está fazendo: Aqui, simulamos o erro que acontece quando o codificador do livro tenta buscar a chave "Hello" no vocabulário. Como a palavra nunca existiu no conto utilizado como treino, o Python aciona um erro do tipo `KeyError`. Em um pipeline de verdade, isso travaria a execução se não houvesse tratamento.

Exemplo Criado

Tentando rodar a palavra "café" em um modelo treinado apenas com textos em inglês. Ela será interpretada como palavra desconhecida.

Tokens Especiais (Special Tokens)

Tradução: Tokens Especiais.

O que é: Tokens artificiais inseridos para controle estrutural do modelo, como marcadores de fim de texto ou palavras desconhecidas.

Função no LLM: Instruem o modelo sobre divisões de frases, palavras inéditas ou preenchimento de lotes de forma organizada.

Relação com outros conceitos: Guardados no Vocabulário com IDs reservados.

Exemplo do Livro

```
# Adicionando os marcadores na lista de palavras únicas
all_tokens = sorted(list(set(preprocessed)))
all_tokens.extend(["<|endoftext|>", "<|unk|>"])
# Criamos o novo vocabulário indexado com esses novos tokens
vocab = {token: integer for integer, token in enumerate(all_tokens)}
print(vocab["<|endoftext|>"]) # Saída: 1130
print(vocab["<|unk|>"]) # Saída: 1131
```

O que o código do livro está fazendo: `all_tokens.extend(...)` insere os tokens estruturais `<|endoftext|>` e `<|unk|>` ao final do vocabulário de palavras conhecidas. Na compreensão de dicionário `{token: integer ...}`, criamos um novo dicionário onde esses tokens ganham os maiores IDs possíveis do vocabulário (índices 1130 e 1131), garantindo que eles tenham números reservados para controle.

Exemplo Criado

Unindo duas notícias diferentes para treino: "Notícia 1... <|endoftext|> Notícia 2..."

Byte Pair Encoding (BPE)

Tradução: BPE.

O que é: Um algoritmo inteligente de tokenização que divide termos desconhecidos em pedaços de palavras conhecidos (subpalavras).

Função no LLM: Resolve o problema de palavras desconhecidas decompondo-as em subunidades sem precisar do `<|unk|>`.

Relação com outros conceitos: Técnica avançada de construção de Vocabulário.

Exemplo do Livro

```
import tiktoken
tokenizer = tiktoken.get_encoding("gpt2")
# 1. Codifica a string desconhecida para obter os IDs do GPT-2
integers = tokenizer.encode("Akwirw ier")
print(integers) # Saída: [33901, 86, 343, 86, 220, 959]
# 2. Decodifica cada ID individualmente
sub_tokens = [tokenizer.decode([i]) for i in integers]
print(sub_tokens) # Saída: ['Ak', 'w', 'ir', 'w', ' ', 'ier']
```

O que o código do livro está fazendo: `tiktoken.get_encoding("gpt2")` carrega o codificador BPE oficial que o GPT-2 usou, que tem um dicionário pré-treinado de 50.257 palavras e subpalavras. `tokenizer.encode("Akwirw ier")` converte a palavra desconhecida em uma lista de 6 números inteiros. A lista compreendida decodifica cada ID em um, provando que o BPE cortou a palavra em 6 partes conhecidas: "Ak", "w", "ir", "w", " " (espaço), e "ier".

Exemplo Criado

```
# Desmembrar "desatualizado" em:  
tokens = ['des', 'atualizado']
```

Janela Deslizante (Sliding Window)

Tradução: Janela Deslizante.

O que é: Técnica de recortar sequências de texto movendo uma moldura de leitura ao longo da frase por um passo definido.

Função no LLM: Cria os conjuntos de dados de treinamento contínuos a partir de um texto bruto longo.

Relação com outros conceitos: O tamanho da janela é o Context Size e o avanço dela é o Stride.

Exemplo do Livro

```
# Visualização de lotes gerados com max_length=4 e stride=1  
batch_1 = [367, 2885, 1464, 1807] # Frase de entrada x  
batch_2 = [2885, 1464, 1807, 3619] # Segunda entrada (deslocada por 1)
```

O que o código do livro está fazendo: Estes dois vetores mostram como a janela deslizante cria amostras de treinamento. No batch_1, temos quatro IDs. Como definimos que o passo (stride) é de 1, a janela anda uma posição para a direita para criar o batch_2: ela remove o primeiro ID (367), mantém os outros três (2885, 1464, 1807) e adiciona o próximo ID do texto (3619).

Exemplo Criado

```
# Sentença: "IA é o futuro" (Contexto=2, Passo=1)  
# Amostra 1: "IA é" -> Alvo: "é o"  
# Amostra 2: "é o" -> Alvo: "o futuro"
```

Pares de Entrada e Alvo (Input-Target Pairs)

Tradução: Pares de Entrada e Alvo.

O que é: O par de dados (X e Y) onde o modelo lê a entrada e tenta adivinhar o alvo que está deslocado de 1 posição para frente.

Função no LLM: Ensinar o modelo a prever o próximo caractere/palavra da frase.

Relação com outros conceitos: Gerados cortando o texto com a Janela Deslizante.

Exemplo do Livro

```
# O código do livro divide a lista de IDs de amostra:  
x = enc_sample[:4] # IDs de entrada: [290, 4920, 2241, 287]  
y = enc_sample[1:4+1] # IDs de alvo (deslocados): [4920, 2241, 287, 257]  
# O loop mostra a relação de adivinhação:  
for i in range(1, 4+1):  
    contexto = x[:i]  
    alvo = x[i] if i < len(x) else 257 # Próximo ID  
# Exemplo de saída: "and" -> "established"
```


O que o código do livro está fazendo: *x* pega os primeiros 4 IDs do texto. *y* pega os mesmos IDs, porém começando do índice 1 até o 5 (deslocando tudo 1 posição para frente). O loop `for` simula o que a rede precisa fazer: ler a palavra no índice 0 ("and") e prever a palavra do alvo no índice 1 ("established"). Depois, acumular o contexto ("and established") e prever o alvo no índice 2 ("himself"), e assim por diante.

Exemplo Criado

```
# Entrada: "A IA está"
# Alvo: "IA está evoluindo"
```

Embedding de Token (Token Embedding)

Tradução: Vetorização / Incorporação de Token.

O que é: A conversão de um ID numérico em uma linha de números decimais (vetor) que guarda o significado da palavra.

Função no LLM: Permite que a rede compute relações e faça cálculos com as palavras.

Relação com outros conceitos: Executado via `torch.nn.Embedding` a partir dos Token IDs.

Exemplo do Livro

```
import torch
torch.manual_seed(123) # Garante que os números aleatórios sejam reprodutíveis
# Cria uma camada com 6 palavras de vocabulário e 3 dimensões de vetor
embedding_layer = torch.nn.Embedding(6, 3)
# Buscamos a representação vetorial do token ID 3
print(embedding_layer(torch.tensor([3])))
# Saída no livro: tensor([[ -0.4015,  0.9666, -1.1481]])
```

O que o código do livro está fazendo: `torch.nn.Embedding(6, 3)` inicializa uma matriz interna com 6 linhas e 3 colunas de números decimais aleatórios (pesos). Ao rodarmos `embedding_layer(torch.tensor([3]))`, o PyTorch não faz nenhuma multiplicação de matrizes complexa: ele simplesmente vai até a tabela de pesos e extrai a linha no índice 3 (a quarta linha da tabela, já que a primeira linha é o índice 0), retornando os valores `[-0.4015, 0.9666, -1.1481]`.

Exemplo Criado

```
# A palavra "cachorro" vira o vetor: [0.12, -0.45, 0.88]
```

Embedding Posicional (Positional Embedding)

Tradução: Embedding Posicional.

O que é: Vetores adicionados à representação da palavra para identificar a ordem que ela ocupa na frase.

Função no LLM: Dá o sentido de ordem gramatical para o modelo (quem é a primeira palavra, segunda, etc.).

Relação com outros conceitos: É somado ao Token Embedding.

Exemplo do Livro

```
context_length = 4 # O modelo suporta sequências de até 4 palavras
output_dim = 256 # Cada vetor terá 256 números de largura
# Criamos a camada posicional
pos_embedding_layer = torch.nn.Embedding(context_length, output_dim)
# Geramos a sequência de posições físicas [0, 1, 2, 3] e extraímos os vetores
pos_embeddings = pos_embedding_layer(torch.arange(context_length))
print(pos_embeddings.shape) # Saída: torch.Size([4, 256])
```

O que o código do livro está fazendo: `torch.arange(context_length)` gera um tensor unidimensional `[0, 1, 2, 3]`, que representa a ordem de leitura física (1ª palavra, 2ª palavra...). Ao passar esse tensor na camada `pos_embedding_layer`, convertemos esses 4 números em 4 vetores de 256 dimensões cada, resultando em um tensor de formato `[4, 256]`. Esses vetores serão somados aos vetores das palavras para indicar a posição de cada uma.

Exemplo Criado

```
# Frase: "Eu vou"
# Vetor final da palavra 'vou' = embedding('vou') + pos_embedding(indice 1)
```

DataLoader

Tradução: Carregador de Dados.

O que é: Um componente que agrupa amostras de dados individuais em blocos (lotes) de tamanho igual para processamento rápido.

Função no LLM: Facilita a alimentação paralela dos dados de treinamento na memória da placa de vídeo (GPU).

Relação com outros conceitos: Agrupa os dados do Dataset.

Exemplo do Livro

```
# Cria o dataloader definindo lote de 8 e contexto de 4 tokens por frase
dataloader = create_dataloader_v1(
    raw_text, batch_size=8, max_length=4, stride=4, shuffle=False
)
# Extraímos o primeiro lote (batch) do carregador
inputs, targets = next(iter(dataloader))
print(inputs.shape) # Saída do livro: torch.Size([8, 4])
```

O que o código do livro está fazendo: `next(iter(dataloader))` converte o `DataLoader` em um iterador do Python e extrai o primeiro lote de informações geradas. `inputs.shape` retorna `torch.Size([8, 4])`, o que nos diz que o `DataLoader` organizou com sucesso os dados de texto em uma matriz com 8 linhas (o batch size) e 4 colunas (o tamanho máximo de contexto das frases).

Exemplo Criado

```
# Juntar dados escolares de alunos de 2 em 2:
# Lote 1: [AlunoA, AlunoB], Lote 2: [AlunoC, AlunoD]
```

Pré-treinamento (Pretraining)

Tradução: Pré-treinamento.

O que é: A fase em que o LLM lê textos em larga escala de forma autônoma para aprender regras gramaticais e estruturas linguísticas fundamentais.

Função no LLM: Cria um modelo genérico básico de compreensão de linguagem.

Relação com outros conceitos: Calibra os valores iniciais da Matriz de Pesos de Embedding.

Exemplo do Livro

Etapa 1: Pré-treinamento (Aprender com dados sem rótulos / next-token prediction)

--> Gera o Foundation Model (Modelo base)

Etapa 2: Fine-tuning (Aprender a classificar ou responder tarefas)

Etapa 3: Alinhamento (Aprender a seguir regras de assistente pessoal)

O que o código do livro está fazendo: O diagrama do livro ilustra que o pipeline de dados de texto implementado no capítulo serve exclusivamente para a Etapa 1 (Pré-treinamento). O modelo treina lendo livros de forma linear, prevendo apenas a próxima palavra a partir do contexto anterior. Ele ainda não sabe responder perguntas (chat) ou seguir instruções; ele está apenas "aprendendo a ler".

Exemplo Criado

Fluxo básico do pré-treinamento:

Input: "O sol brilha no..." -> Alvo real: "céu"

Perda = Calcular a diferença entre a predição da rede e o ID real da palavra "céu".

Predição da Próxima Palavra (Next-Word Prediction Task)

Tradução: Predição da Próxima Palavra.

O que é: O objetivo de aprendizado do modelo: tentar adivinhar a próxima palavra dada uma sequência anterior de contexto.

Função no LLM: É o mecanismo que obriga o modelo a aprender semântica e sintática tentando diminuir a taxa de erro.

Relação com outros conceitos: O modelo tenta prever os elementos contidos no tensor de Alvo (Target).

Exemplo do Livro

[LLMs] -----> target: learn

[LLMs, learn] -----> target: to

[LLMs, learn, to] -----> target: predict

[LLMs, learn, to, predict] -----> target: one

O que o código do livro está fazendo: Mostra graficamente como a rede aprende de forma acumulativa. A entrada cresce palavra por palavra à medida que avançamos na frase. Ao calcular o erro das previsões, a rede ajusta seus pesos internos para que a palavra predita coincida com a palavra real que está na sequência.

Exemplo Criado

Digitar "Estou com muito" no teclado e o celular sugerir "sono" ou "fome".

Word2Vec

Tradução: Word2Vec.

O que é: Um modelo clássico de embedding estático onde cada palavra tem um vetor fixo atribuído na biblioteca de termos.

Função no LLM: Mostrou que é possível agrupar termos parecidos matematicamente.

Relação com outros conceitos: É a versão estática e não contextual de um Token Embedding.

Exemplo do Livro

```
# Conceito de distância geométrica no espaço de embeddings do Word2Vec:  
dist_aves = torch.dist(vetor("eagle"), vetor("duck")) # Distância curta (ex: 0.2)  
dist_cidade = torch.dist(vetor("eagle"), vetor("Berlin")) # Distância longa (ex: 1.5)
```

O que o código do livro está fazendo: Explica que em um espaço vetorial bem treinado, a distância matemática (euclidiana) entre vetores de conceitos semelhantes ("eagle" e "duck", que são aves) é muito menor do que a distância entre conceitos não relacionados ("eagle" e "Berlin", que é uma cidade alemã). O modelo de embeddings usa geometria para organizar significados.

Exemplo Criado

```
# Mapear cores: vermelho e rosa têm coordenadas muito próximas em RGB; verde fica longe.
```

Espaço de Alta Dimensionalidade (High-Dimensional Space)

Tradução: Espaço de Alta Dimensionalidade.

O que é: A estrutura dimensional onde os vetores de embedding estão posicionados.

Função no LLM: Permite expressar relações muito ricas entre os dados (como tempo verbal, gênero, pluralidade, emoção, categoria).

Relação com outros conceitos: Define a dimensão da coluna dos Token Embeddings.

Exemplo do Livro

```
# Definição das constantes de largura vetorial usadas no livro  
GPT2_embedding_dim = 768  
GPT3_embedding_dim = 12288
```

O que o código do livro está fazendo: Estas variáveis definem a quantidade de eixos coordenados que o espaço vetorial possui. No GPT-2 básico, cada palavra é mapeada em um ponto geométrico que possui 768 coordenadas decimais diferentes. No GPT-3 grande, são 12.288 coordenadas, o que dá à rede uma capacidade gigantesca de guardar informações semânticas sutis.

Exemplo Criado

```
# Classificar livros com 200 critérios diferentes (suspense, páginas, autor, etc.)  
# para guardá-los em uma estante conceitual.
```

Embeddings Posicionais Absolutos vs. Relativos

Tradução: Embeddings Posicionais Absolutos vs. Relativos.

O que é: Métodos para informar ao modelo em qual posição a palavra se encontra na frase.

Função no LLM: Evita que a frase seja interpretada de forma desordenada (como um saco de palavras soltas).

Relação com outros conceitos: Variantes da técnica de Positional Embedding.

Exemplo do Livro

```
# GPT utiliza embeddings absolutos indexados por números inteiros:  
posicao_da_palavra = 0 # 1ª palavra da frase  
vetor_posicao_0 = pos_embedding_layer(torch.tensor([posicao_da_palavra]))
```

O que o código do livro está fazendo: Demonstra que na abordagem de embeddings absolutos do GPT, cada índice de vaga na frase (como a vaga 0 para a primeira palavra) tem seu próprio vetor de características exclusivo na camada de pesos. Esse vetor é obtido fazendo uma busca pelo índice físico da palavra, independentemente de qual palavra esteja ocupando aquela vaga.

Exemplo Criado

```
# Absoluto: Dizer "Seu armário é o número 3".  
# Relativo: Dizer "Seu armário está a duas posições do armário da recepção".
```

Codificação One-Hot (One-Hot Encoding)

Tradução: Codificação One-Hot.

O que é: Representar uma palavra por uma lista binária cheia de zeros, contendo apenas um número 1 indicando a sua posição na lista.

Função no LLM: É o equivalente matemático à camada de embeddings. Como multiplicar matrizes por vetores cheios de zeros é muito ineficiente, usamos o nn.Embedding que funciona apenas como um "localizador" (lookup) rápido de linhas.

Relação com outros conceitos: É a técnica pesada que os Token Embeddings otimizam por baixo do capô.

Exemplo do Livro

```
# 1. Criamos o vetor One-Hot de tamanho 6 representando o ID 3:  
one_hot = torch.tensor([0.0, 0.0, 0.0, 1.0, 0.0, 0.0])  
# 2. Peso da camada de embeddings (matriz 6x3)  
pesos = embedding_layer.weight  
# 3. Fazemos a multiplicação de matrizes (@ no PyTorch)  
vetor_resultado = one_hot @ pesos  
print(vetor_resultado) # Retorna exatamente o vetor da linha 3 dos pesos.
```

O que o código do livro está fazendo: `one_hot = ...` cria uma lista com um único valor 1.0 na quarta posição (índice 3). `one_hot @ pesos` faz uma multiplicação matricial. Como todas as posições são 0, exceto a de índice 3, todas as linhas da matriz de pesos são multiplicadas por zero (anuladas), e apenas a linha 3 é multiplicada por 1. Isso prova matematicamente que extrair uma linha de forma direta é exatamente equivalente a fazer uma multiplicação de matriz por um vetor One-Hot.

Exemplo Criado (Capítulo 2, pág. 43)

Um controle com 4 botões de luz onde apenas um botão de cada vez fica apertado.

Matriz de Pesos de Embedding (Embedding Weight Matrix)

Tradução: Matriz de Pesos de Embedding.

O que é: A tabela interna contendo os parâmetros da camada de embeddings de formato [Vocab Size, Embedding Dim].

Função no LLM: Armazena os vetores contínuos modificáveis de cada uma das palavras durante o aprendizado do modelo.

Relação com outros conceitos: É a estrutura consultada pelo ID do token para obter o Token Embedding.

Exemplo do Livro

```
# Exibe a tabela interna de parâmetros que a camada de embedding gerou
print(embedding_layer.weight)
# Saída do livro:
# Parameter containing:
# tensor([[[ 0.3374, -0.1778, -0.1690], ...], requires_grad=True)
```

O que o código do livro está fazendo: `.weight` extrai o parâmetro interno da camada PyTorch. A matriz tem formato [6, 3] (6 palavras no vocabulário, 3 coordenadas de embedding por palavra). O aviso `requires_grad=True` sinaliza ao PyTorch que esses coeficientes não são fixos: eles serão modificados de forma automática durante a retropropagação (backpropagation) para melhorar o desempenho do modelo conforme ele erra as previsões.

Exemplo Criado

Uma caderneta onde cada linha tem o nome de uma cor e suas três coordenadas de mistura RGB.

Passo da Janela (Stride)

Tradução: Passo da Janela.

O que é: A quantidade de tokens que a moldura de leitura desloca-se a cada novo recorte de amostras.

Função no LLM: Controla a sobreposição e a repetição de dados de treino no pipeline de dados.

Relação com outros conceitos: Parâmetro central do Dataset na técnica de Sliding Window.

Exemplo do Livro

```
# Se os tokens são: [0, 1, 2, 3, 4, 5, 6, 7]
# Se max_length = 4 e stride = 4:
# Lote 1: [0, 1, 2, 3] -> (Pula 4 tokens) -> Lote 2: [4, 5, 6, 7]
```

O que o código do livro está fazendo: Mostra o impacto prático do parâmetro `stride`. Ao igualar o `stride` ao comprimento da janela `max_length = 4`, a janela de leitura avança exatamente quatro posições para o lado. Isso evita qualquer sobreposição ou repetição de palavras entre os exemplos de treino consecutivos, otimizando o treinamento contra o sobreajuste (overfitting).

Exemplo Criado

Uma pessoa caminhando: dar passos curtos de 20cm versus passos normais de 70cm.

Transmissão de Tensores (Broadcasting)

Tradução: Transmissão (Broadcasting) de Tensores.

O que é: Um recurso matemático que estica de forma virtual as dimensões de um tensor menor para que ele possa ser somado a um maior.

Função no LLM: Permite somar o mesmo vetor de posições [Context Length, Embedding Dim] a todas as frases do lote de formato [Batch Size, Context Length, Embedding Dim] de forma eficiente.

Relação com outros conceitos: Permite unir o Token Embedding com o Positional Embedding.

Exemplo do Livro

```
# token_embeddings.shape -> torch.Size([8, 4, 256])
# pos_embeddings.shape -> torch.Size([4, 256])
# Realiza a soma de tensores com formatos diferentes
input_embeddings = token_embeddings + pos_embeddings
print(input_embeddings.shape) # Saída: torch.Size([8, 4, 256])
```

O que o código do livro está fazendo: Temos um tensor tridimensional com 8 lotes ([8, 4, 256]) e tentamos somar a ele um tensor bidimensional ([4, 256]). O PyTorch percebe que as dimensões finais coincidem (4 e 256) e, por meio do mecanismo de Broadcasting, replica virtualmente o tensor posicional 8 vezes (uma vez para cada frase do lote) para realizar a soma matemática elemento por elemento sem duplicar os dados fisicamente na memória RAM.

Exemplo Criado

```
# Aplicar um carimbo de "Aprovado" no canto da página em um lote com 50 documentos.
```